Cloud for AI: Deel 2

Auteur: Joussef Boulbane

Dataset: AI4I 2020 Predictive Maintenance (10.000 rijen)

Repository: github.com/JoussefB/cloud-for-ai

Modelvergelijking: Baseline vs. Challenger

Voor dit project heb ik twee modellen getraind en geëvalueerd op een gestratificeerde testset van 2.000 rijen (waarvan 68 effectieve defecten):

- **Model 1 (Baseline): Logistic Regression.** Dit model maakt gebruik van L2-regularisatie en `class_weight='balanced'`. Het fungeert als een doelbewust eenvoudig, transparant en snel te trainen startpunt.
- **Model 2 (Challenger): Random Forest Classifier.** Dit is een complexer model met 200 beslissingsbomen dat in staat is om niet-lineaire interacties tussen sensoren (bijvoorbeeld de combinatie van koppel en rotatiesnelheid) te vangen.

Wat ze scoorden

Beide modellen zijn geëvalueerd met een verlaagde classificatiedrempelwaarde van 0.15:

Model	Recall	Precision	Gemiste Defecten (FN)	Valse Alarmen (FP)	Netto Winst
Logistic Regression (Baseline)	95.6% (65/68)	6.2%	3	979	€503.150
Random Forest (Gekozen)	**91.2% (62/68)**	**34.8%**	**6**	**116**	**€602.600**

Welk model ik houd en waarom

Ik kies beslist voor de **Random Forest** voor uiteindelijke deployment. Hoewel Logistic Regression op papier een ruwe recall heeft die fracties hoger ligt (95,6% vs 91,2%), zijn de praktische argumenten voor Random Forest doorslaggevend:

- **Voorkomen van Alert Fatigue:** De baseline genereert 979 valse alarmen op 1.932 gezonde gevallen. Technici zouden bedolven worden onder loze meldingen en het systeem gaan negeren. Random Forest reduceert het aantal valse meldingen met meer dan 88% (van 979 naar 116), waardoor gerichte actie mogelijk blijft.
- **Economisch voordeel:** Doordat het Random Forest model onnodige inspectiekosten drastisch verlaagt, levert het over 2.000 cycli direct €99.450 méér netto economisch voordeel op dan de baseline.
- **Realtime Snelheid:** De gemiddelde inferentietijd per machine bedraagt 0,85 ms, wat ruim binnen de vereiste limiet van 1 seconde (near-realtime monitoring) valt.

Data Leakage & Eerlijke Train/Test-Split

In deze dataset zat een levensgroot risico op data leakage verstopt in de vijf diagnostische sub-storingskolommen: TWF, HDF, PWF, OSF en RNF. Omdat deze kolommen deterministische deelcomponenten zijn van de doelvariabele 'Machine failure', zou het model een triviale identiteitsfunctie met 100% target leakage leren als ze werden meegenomen. In een echte fabriek weet je deze specifieke faalmechanismen bovendien pas achteraf na demontage, nooit vooraf tijdens monitoring. Deze kolommen zijn daarom strikt geëxcludeerd uit de configuratie. Een eerlijke split is daarnaast gegarandeerd door de data hermetisch te scheiden met `stratify=y`, zodat de 3,39% defecten exact gelijk verdeeld zijn over de trainings- en testset. Cruciaal is dat alle schaling en encodering veilig zijn ingebouwd in een Scikit-learn Pipeline. De `StandardScaler` en `OneHotEncoder` worden uitsluitend gefit op de trainingsdata en pas daarna blind toegepast op de testset, waardoor het wiskundig onmogelijk is dat er informatie van de testset naar de modelparameters lekt.